UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería, Matemática y Ciencias Físicas

Curso: Aseguramiento de la Calidad

PROYECTO FINAL

Plan de Aseguramiento de la Calidad, Pruebas Manuales y Automatización con Playwright y TypeScript sobre la tienda OpenCart Demo (Abstracta)

Grupo 10 — Variante 2

Integrantes

Nombre	Carné	Rol
Teddy Leonardo Hernández Pérez	1790-22-2563	Coordinación y automatización
Gelen Dayanna López Morales	1790-21-14904	Plan QA, documentación y trazabilidad
Milton Adrián Martínez Ávila	1790-22-9129	Pruebas manuales, exploratorias y defectos

Guatemala, 24 de octubre de 2026

ÍNDICE

1. Introducción....4

2. Objetivos....4

2.1 Objetivo general....4

2.2 Objetivos específicos....5

3. Marco teórico....5

3.1 Calidad de software y aseguramiento de la calidad....5

3.2 Pruebas de software y niveles de prueba....6

3.3 Técnicas de diseño de casos de prueba....6

3.4 Pruebas exploratorias....6

3.5 Automatización de pruebas y Page Object Model....6

3.6 Pruebas inestables (flaky tests)....7

3.7 Gestión de defectos....7

4. Descripción del sistema evaluado....7

5. Alcance y metodología de aseguramiento de la calidad....8

5.1 Funcionalidades en alcance....8

5.2 Fuera de alcance....8

5.3 Metodología....8

5.4 Criterios de entrada y salida....8

6. Planificación y organización del equipo....9

6.1 Roles y responsabilidades....9

6.2 Organización del trabajo y control de versiones....9

7. Diseño de casos de prueba....10

7.1 Aplicación de las técnicas....10

8. Ejecución de pruebas manuales....10

8.1 Pruebas exploratorias 11
9. Automatización con Playwright y TypeScript 11
10. Arquitectura de pruebas: POM, fixtures y configuración 12
10.1 Page Object Model 12
10.2 Fixtures, helpers y datos de prueba 12
10.3 Configuración 12
11. Resultados por navegador 12
12. Defectos identificados y seguimiento 14
13. Matriz de trazabilidad 15
14. Evidencias de ejecución 15
15. Análisis de resultados y estabilidad 15
16. Dificultades y soluciones 16
17. Conclusiones individuales 17
17.1 Teddy Leonardo Hernández Pérez 17
17.2 Gelen Dayanna López Morales 17
17.3 Milton Adrián Martínez Ávila 17
18. Recomendaciones 18
19. Referencias 18
Anexos 20
Anexo A. Enlaces del proyecto 20
Anexo B. Capturas de ejecución 20
Anexo C. Evidencias manuales y de defectos 23

1. Introducción

El presente documento constituye el informe final del Proyecto Final del curso de Aseguramiento de la Calidad de la Universidad Mariano Gálvez de Guatemala, desarrollado por el Grupo 10, al que se asignó la Variante 2: la tienda en línea OpenCart Demo publicada por Abstracta en https://opencart.abstracta.us/. El proyecto tiene como propósito aplicar de forma integral los conceptos del curso: planificación de pruebas, diseño de casos con técnicas formales, ejecución de pruebas manuales y exploratorias, automatización con Playwright y TypeScript, gestión de defectos con evidencias y GitHub Issues, y elaboración de un reporte profesional de resultados.

A diferencia de los demás grupos, el Grupo 10 está conformado por tres integrantes, por lo que el enunciado amplía su alcance: la suite automatizada debe ejecutarse también en WebKit (además de Chromium y Firefox), deben diseñarse al menos 23 casos de prueba manuales y documentarse un mínimo de 7 defectos. Este informe describe lo realizado, presenta los resultados verificables que se encuentran en el repositorio del proyecto y señala con claridad los elementos que, a la fecha de redacción, permanecen pendientes.

Como criterio editorial, el documento se rige por una regla de veracidad: únicamente se reportan resultados, ejecuciones y evidencias que existen en el repositorio https://github.com/leotedd/Proyecto-QA-Tienda. Cuando un elemento no ha sido ejecutado o no cuenta con evidencia, se indica explícitamente como pendiente, sin presentar como aprobado algo que no fue verificado.

2. Objetivos

2.1 Objetivo general

Desarrollar un plan de aseguramiento de la calidad completo para la tienda OpenCart Demo (Abstracta), que incluya casos de prueba manuales documentados, una suite de automatización funcional con Playwright y TypeScript, la gestión de defectos con evidencias y GitHub Issues, y un reporte profesional de resultados.

2.2 Objetivos específicos

- Diseñar un plan de pruebas que cubra las funcionalidades críticas F01 a F09 definidas en el enunciado.
- Elaborar casos de prueba aplicando partición de equivalencia, análisis de valores en la frontera y tablas de decisión.
- Implementar una suite automatizada con Playwright y TypeScript siguiendo el patrón Page Object Model, con fixtures reutilizables y pruebas parametrizadas.
- Ejecutar la suite en Chromium, Firefox y WebKit, organizada con las etiquetas @smoke y @regression para ejecución selectiva.
- Documentar y clasificar los defectos por severidad y prioridad, registrándolos como GitHub Issues.
- Analizar la estabilidad de la suite e identificar pruebas inestables, documentando el diagnóstico.
- Mantener la trazabilidad entre funcionalidades, casos manuales, pruebas automatizadas, evidencias y defectos.

3. Marco teórico

3.1 Calidad de software y aseguramiento de la calidad

La calidad de software puede entenderse como el grado en que un producto satisface los requisitos especificados y las necesidades de sus usuarios. Galin (2004) distingue entre el control de calidad, orientado a detectar defectos en productos concretos, y el aseguramiento de la calidad, que comprende el conjunto sistemático de actividades planificadas para proporcionar confianza en que el proceso y el producto cumplen los requisitos. En este proyecto, el aseguramiento se materializa en un plan de pruebas, criterios de entrada y salida, gestión formal de defectos y trazabilidad.

Cantú (2011) plantea que la calidad no es un atributo que se agrega al final, sino el resultado de una cultura organizacional orientada a la mejora continua. Esa visión justifica que el equipo haya trabajado por fases, con control de versiones y revisiones mediante Pull Requests, en lugar de concentrar las actividades de verificación al cierre del proyecto.

3.2 Pruebas de software y niveles de prueba

Según Kaner, Falk y Nguyen (1999), probar software es ejecutar un programa con la intención de encontrar errores, y una buena prueba es aquella con alta probabilidad de revelar un defecto aún no descubierto. Naik y Tripathy (2008) clasifican las pruebas en niveles (unidad, integración, sistema y aceptación) y en tipos (funcionales y no funcionales). El presente proyecto se ubica en el nivel de pruebas de sistema, de tipo funcional y de caja negra, ya que el equipo no tiene acceso al código fuente de la aplicación y evalúa su comportamiento observable desde la interfaz web.

3.3 Técnicas de diseño de casos de prueba

La partición de equivalencia divide el dominio de entradas en clases cuyos elementos deberían provocar el mismo comportamiento, de modo que basta un representante por clase. El análisis de valores en la frontera complementa la técnica anterior al seleccionar valores en los límites de cada clase, donde se concentra una proporción elevada de defectos. Las tablas de decisión modelan combinaciones de condiciones y las acciones esperadas, y son útiles cuando el resultado depende de varias condiciones simultáneas (Toledo Rodríguez et al., 2014). En el proyecto, estas técnicas se aplicaron, por ejemplo, a los campos obligatorios vacíos del registro, a la longitud mínima del texto de reseña (25 a 1000 caracteres) y a la combinación entre la aceptación de la política de privacidad y la validez de los datos de registro.

3.4 Pruebas exploratorias

Las pruebas exploratorias combinan aprendizaje, diseño y ejecución simultáneos. El probador utiliza su conocimiento del sistema para guiar la exploración y descubrir comportamientos que los casos formales no contemplan. El enunciado exige al menos 30 minutos de exploración, y fue precisamente durante esta actividad (EXP-05) donde se confirmó el defecto DR-001 en una segunda ruta de la aplicación.

3.5 Automatización de pruebas y Page Object Model

La automatización de pruebas consiste en utilizar software para ejecutar casos de prueba y comparar resultados reales con esperados de forma repetible. Playwright es un marco de código abierto mantenido por Microsoft que permite automatizar Chromium, Firefox y WebKit con una única API, e incluye espera automática (auto-waiting), aserciones web que se

reintentan, aislamiento por contexto de navegador, fixtures, proyectos multinavegador, etiquetas y reportes HTML (Microsoft, 2026). El patrón Page Object Model encapsula cada página de la aplicación en una clase que expone localizadores y acciones, de modo que las pruebas expresan intención de negocio y los cambios de la interfaz se corrigen en un solo lugar.

3.6 Pruebas inestables (flaky tests)

Una prueba inestable es aquella que, sin cambios en el código ni en la aplicación, a veces pasa y a veces falla. Sus causas más comunes son esperas fijas, localizadores frágiles, dependencia del orden o del estado compartido y variaciones de latencia del entorno. Es fundamental distinguir una prueba inestable de un defecto real reproducible: este último falla de forma consistente y debe reportarse, no ocultarse.

3.7 Gestión de defectos

Un defecto es una imperfección del producto que puede causar que el sistema no cumpla su función. Su gestión incluye el registro con pasos reproducibles, la clasificación por severidad (impacto técnico) y prioridad (urgencia de corrección), la evidencia y el seguimiento de su ciclo de vida. El enunciado exige que cada defecto se registre además como GitHub Issue, con estado coincidente.

4. Descripción del sistema evaluado

OpenCart es una plataforma de comercio electrónico de código abierto escrita en PHP. La instancia evaluada, OpenCart Demo de Abstracta (https://opencart.abstracta.us/), es un entorno público de práctica compartido por varios grupos del curso. Ofrece un catálogo de productos organizado por categorías (Desktops, Laptops & Notebooks, Components, Tablets, Software, Phones & PDAs, Cameras y MP3 Players), un buscador en el encabezado, páginas de detalle con pestañas de descripción y reseñas, y un módulo de cuenta de usuario con registro, inicio y cierre de sesión, edición de datos personales y cambio de contraseña.

Al tratarse de un entorno compartido, el equipo atendió las consideraciones éticas del enunciado: se usaron únicamente datos ficticios (correos con el dominio reservado example.com y contraseñas de prueba), cuentas desechables generadas por la suite y un volumen moderado de reseñas, sin intentar acceder a cuentas de otros grupos.

5. Alcance y metodología de aseguramiento de la calidad

5.1 Funcionalidades en alcance

ID	Funcionalidad	Descripción
F01	Registro de usuario	Creación de cuenta nueva con datos válidos e inválidos
F02	Inicio de sesión	Acceso con credenciales válidas e inválidas
F03	Cierre de sesión	Finalización de sesión y verificación de la redirección
F04	Edición de perfil	Actualización de nombre, apellido, email y teléfono
F05	Cambio de contraseña	Modificación de la contraseña de la cuenta
F06	Calificación de producto	Calificación por estrellas (1 a 5)
F07	Reseña de producto	Comentario de texto sobre un producto
F08	Navegación y búsqueda	Categorías y buscador del catálogo
F09	Manejo de errores y validaciones	Formularios incompletos, entradas inválidas y duplicados

5.2 Fuera de alcance

Quedan fuera del proyecto las pruebas de rendimiento, las pruebas de seguridad o penetración, la infraestructura del servidor, el proceso de compra o checkout completo y cualquier funcionalidad administrativa no expuesta al usuario final.

5.3 Metodología

La estrategia combinó tres enfoques complementarios. Primero, pruebas manuales diseñadas a partir de las funcionalidades del alcance y registradas en la plantilla de casos de prueba. Segundo, una sesión de pruebas exploratorias de aproximadamente 30 minutos. Tercero, una suite automatizada que cubre los flujos críticos mediante pruebas smoke y de regresión. Los defectos se documentan con la plantilla de reporte de errores y se registran como GitHub Issues.

5.4 Criterios de entrada y salida

Criterios de entrada: la aplicación está accesible; el grupo cuenta con usuarios de prueba documentados; el entorno tiene Node.js, Playwright y Visual Studio Code configurados. Criterios de salida: el 100 % de los casos ejecutados con resultado registrado; ningún defecto crítico sin resolver o sin justificación; la suite se ejecuta sin errores en los navegadores requeridos; el reporte de defectos está completo con sus Issues.

6. Planificación y organización del equipo

6.1 Roles y responsabilidades

Integrante	Rol	Contribuciones documentadas en el repositorio
Teddy Leonardo Hernández Pérez	Coordinación y automatización	Estructura inicial del repositorio; suite Playwright con POM, fixtures y helpers; ejecución multinavegador; reporte de ejecución automatizada; casos automatizados en Excel; video individual; revisión e integración de Pull Requests
Gelen Dayanna López Morales	Plan QA, documentación y trazabilidad	Plan de aseguramiento de la calidad; casos funcionales F01–F05 y adicionales F08; matriz de trazabilidad; análisis documental de estabilidad
Milton Adrián Martínez Ávila	Pruebas manuales, exploratorias y defectos	10 casos manuales F06–F09 ejecutados con evidencia; pruebas exploratorias; reporte del defecto DR-001; video de ejecución manual

6.2 Organización del trabajo y control de versiones

El trabajo se organizó en ramas por integrante (feature/teddy, feature/dayanna y feature/martinez132), integradas a la rama principal mediante Pull Requests. El historial del repositorio registra commits desde el 1 de octubre de 2026 (configuración inicial) hasta el 24 de octubre de 2026, incluidos los Pull Requests #1, #2, #4, #5, #6 y #7. El historial de la rama principal supera los 10 commits exigidos.

Semana	Actividades registradas en el repositorio
1 (29 sep – 3 oct)	Configuración de la estructura inicial del proyecto QA (1/10/2026).
2 (4 – 10 oct)	Casos manuales F06–F09 y evidencias (Milton Adrián Martínez Ávila, 9/10); suite automatizada con POM y fixtures (Teddy Leonardo Hernández Pérez, PR #2).
3 (11 – 17 oct)	Ejecución de la suite en Chromium, Firefox y WebKit (17/10/2026).
4 (18 – 24 oct)	Pruebas exploratorias y video (Milton Adrián Martínez Ávila); Plan QA, casos F01–F05, trazabilidad y análisis de estabilidad (Gelen Dayanna López Morales); reporte de ejecución, evidencias, video y casos automatizados (Teddy Leonardo Hernández Pérez).

Nota: la Guía para el trabajo grupal solicita acta de constitución, cronograma WBS, minutas y política de trabajo. En la carpeta de documentación existen los archivos de plantilla proporcionados por el catedrático; no se encontraron versiones completadas por el equipo, por lo que no se presentan como trabajo realizado.

7. Diseño de casos de prueba

Los casos se diseñaron con base en la lista sugerida por el enunciado y en las técnicas descritas en el marco teórico. Cada caso registra identificador, funcionalidad, título, precondición, pasos, datos de prueba, resultado esperado, resultado obtenido, estado y evidencia.

Responsable	Archivo	Casos	Estado
Milton Adrián Martínez Ávila	Casos_Prueba_Martinez_F06-F09.xlsx	10 (TC-V01–V03, TC-C01–C03, TC-N01–N04)	Ejecutados: 9 Pass, 1 Fail
Gelen Dayanna López Morales	casos-funcionales-F01-F05.md	13 (TC-R01–R04, TC-L01–L04, TC-P01–P03, TC-N05, TC-N06)	Diseñados; pendientes de ejecución
Teddy Leonardo Hernández Pérez	Casos_Prueba_Hernandez_Automatizacion.xlsx	25 casos automatizados (5 smoke, 20 regresión)	Ejecutados: 24 Pass, 1 Fail (DR-001)

7.1 Aplicación de las técnicas

Técnica	Aplicación en el proyecto
Partición de equivalencia	Login con contraseña incorrecta, correo no registrado y credenciales vacías; registro con correo duplicado; reseñas válidas frente a inválidas.
Valores en la frontera	Campos obligatorios vacíos en registro y reseña; texto de reseña vacío frente al mínimo de 25 caracteres; confirmación de contraseña distinta.
Tablas de decisión	Registro: política de privacidad aceptada o no, combinada con datos válidos o inválidos.

8. Ejecución de pruebas manuales

Los casos F06 a F09 fueron ejecutados por Milton Adrián Martínez Ávila con evidencia en evidencias/casos-manuales/martinez/. Los resultados registrados son los siguientes:

ID	Caso	Estado	Resultado obtenido (resumen)
TC-V01	Calificar con 5 estrellas y enviar reseña	Pass	Reseña de MacBook aceptada con mensaje de confirmación
TC-V02	Reseña sin calificación	Pass	Advertencia: seleccionar una valoración
TC-V03	Calificar el mismo producto dos veces	Pass	Segunda reseña aceptada con confirmación
TC-C01	Reseña válida	Pass	Reseña de iPhone aceptada
TC-C02	Reseña sin texto	Pass	Advertencia: texto entre 25 y 1000 caracteres
TC-C03	Reseña con caracteres especiales	Pass	Procesada correctamente
TC-N01	Navegar por categorías	Pass	Tablets mostró Samsung Galaxy Tab 10.1
TC-N02	Buscar producto por nombre	Pass	Resultados de MacBook correctos
TC-N03	Formulario con campos vacíos	Pass	Validación de valoración obligatoria

ID	Caso	Estado	Resultado obtenido (resumen)
TC-N04	Caracteres especiales en búsqueda	Fail	Muestra "Tu carrito de la compra está vacío" (DR-001)

Los 13 casos de F01–F05 y los adicionales de F08 están diseñados en documentacion/casos-prueba/casos-funcionales-F01-F05.md, pero a la fecha de este informe su estado es PENDIENTE: no existen resultados ni evidencias de su ejecución manual en el repositorio. Las funcionalidades F01–F05 sí fueron verificadas por la suite automatizada (sección 10).

8.1 Pruebas exploratorias

El 22/10/2026 Milton Adrián Martínez Ávila realizó una sesión exploratoria de aproximadamente 30 minutos sobre F06–F09. El hallazgo EXP-05 consistió en ingresar a la categoría Software, que no contiene productos: el sistema mostró "¡Tu carrito de la compra está vacío!" en lugar de un mensaje de categoría sin productos. El comportamiento es reproducible y se relacionó con DR-001.

9. Automatización con Playwright y TypeScript

La suite se implementó con Playwright Test y TypeScript, y se organiza en tres archivos de especificación. Las 25 pruebas por navegador se etiquetan con @smoke o @regression, lo que permite ejecutar subconjuntos mediante los scripts npm run test:smoke y npm run test:regression.

Suite	Archivo	Pruebas	Cobertura
Smoke (@smoke)	tests/smoke.spec.ts	5	Inicio, búsqueda (TC-N02), categoría (TC-N01), detalle con formulario de reseña, login
Regresión auth (@regression)	tests/regression-auth.spec.ts	10	Registro (5 escenarios), login válido, 3 logins rechazados parametrizados, cierre de sesión
Regresión reseñas (@regression)	tests/regression-reviews.spec.ts	10	Reseñas válidas e inválidas parametrizadas, TC-V03, TC-N04, perfil y contraseña
Total por navegador		25	

10. Arquitectura de pruebas: POM, fixtures y configuración

10.1 Page Object Model

La carpeta pages/ contiene cinco Page Objects: HomePage (buscador, categorías, resultados), LoginPage (formulario de inicio de sesión), RegisterPage (formulario de registro y validaciones por campo), AccountPage (menú de cuenta, edición de perfil, cambio de contraseña y cierre de sesión) y ProductPage (detalle de producto y formulario de reseña). Las pruebas no contienen selectores directos; invocan métodos de estas clases, lo que mejora la legibilidad y el mantenimiento.

10.2 Fixtures, helpers y datos de prueba

El archivo fixtures/index.ts extiende el objeto test de Playwright con un fixture por Page Object, un usuario registrado por worker (registeredUser), una página de cuenta con sesión iniciada (loggedInAccountPage) y un usuario nuevo por prueba (newUser). El archivo helpers/auth.ts concentra las funciones registerUser, createAccount y loginAs. Los datos se centralizan en test-data/users.ts y test-data/reviews.ts; los usuarios se generan con correos únicos del dominio example.com. Nueve de las 25 pruebas son parametrizadas a partir de arreglos de datos (invalidLoginCases, validReviewCases e invalidReviewCases), y la suite de registro usa un hook beforeEach.

10.3 Configuración

El archivo playwright.config.ts define la URL base de la aplicación, tres proyectos (chromium con Desktop Chrome, firefox con Desktop Firefox y webkit con Desktop Safari), capturas de pantalla solo ante fallos, trazas en el primer reintento y el reporter HTML. El repositorio incluye un archivo .npmrc con ignore-scripts=true, por lo que los navegadores se instalan explícitamente con npx playwright install. La configuración vigente no declara retries, timeout ni video; la sección de recomendaciones propone alinearla con el ejemplo del enunciado.

11. Resultados por navegador

La suite completa se ejecutó el 17/10/2026 en Windows 11, con Node.js 24.14.1, Playwright 1.63.0 y sin reintentos. Los resultados se respaldan con capturas de terminal y del reporte HTML en evidencias/automatizadas/teddy/.

Navegador	Comando	Pruebas	Passed	Failed	Flaky	Duración
Chromium	npx playwright test --project=chromium	25	25	0	0	1.4 min
Firefox	npx playwright test --project=firefox	25	25	0	0	1.6 min
WebKit	npx playwright test --project=webkit	25	25	0	0	2.0 min
Total		75	75	0	0	

Etiqueta	Comando	Navegador	Pruebas	Resultado
@smoke	npm run test:smoke -- --project=chromium	Chromium	5	5 passed (32.4 s)
@regression	npm run test:regression -- -- project=chromium	Chromium	20	20 passed (1.2 min)

El 24/10/2026 se realizó una verificación previa a la entrega: la compilación TypeScript (npx tsc --noEmit) no presentó errores y la suite smoke en Chromium obtuvo 5 pruebas aprobadas en 28.1 segundos.

Sobre TC-N04: la prueba está marcada con test.fail() porque reproduce el defecto DR-001. Playwright la contabiliza como aprobada mientras falle exactamente en la aserción esperada (Expected: "There is no product that matches the search criteria." / Received: "Your shopping cart is empty!"). Si la aplicación se corrige, la prueba aparecerá como fallida y deberá retirarse la marca. Esta decisión evita ocultar el defecto y, a la vez, mantiene la suite en verde para los demás escenarios.

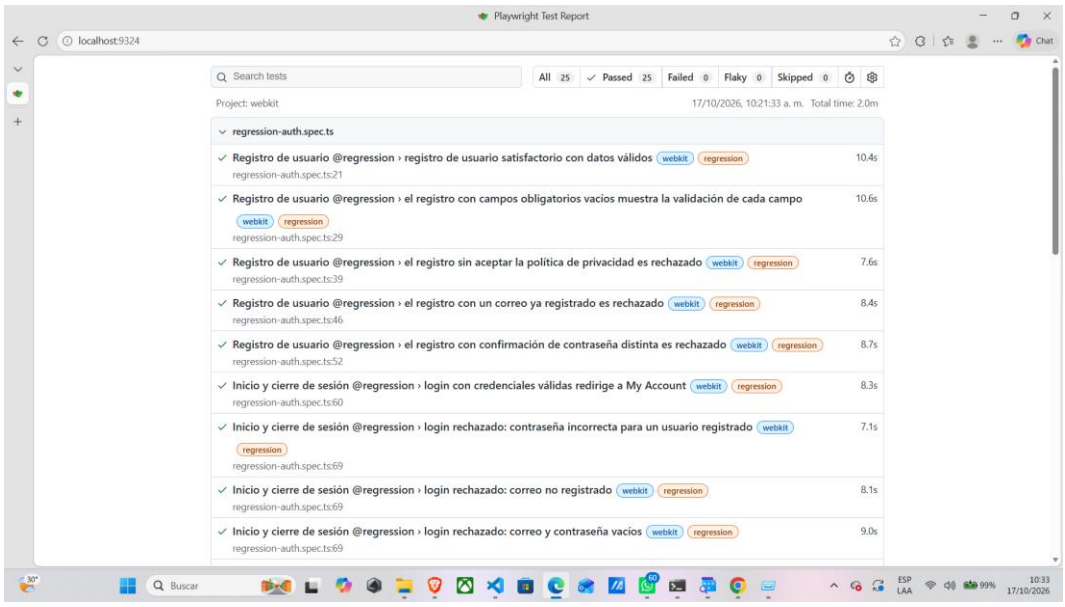


Figura 1. Reporte HTML de Playwright para WebKit: 25 pruebas aprobadas (17/10/2026).

12. Defectos identificados y seguimiento

Campo	DR-001
Título	La búsqueda sin coincidencias muestra el mensaje del carrito vacío en lugar de un mensaje de búsqueda
Fecha / descubierto por	09/10/2026 — Milton Adrián Martínez Ávila
Funcionalidad / caso origen	F09 — Manejo de errores y validaciones / TC-N04
Tipo	Manual (confirmado también en exploratoria EXP-05 y en la suite automatizada)
Severidad / prioridad	Baja / P3
Ambiente	https://opencart.abstracta.us/ — Microsoft Edge (Chromium) — Windows
Pasos	1. Ir al sitio. 2. Ubicar el buscador. 3. Ingresar /"'&%#@!. 4. Ejecutar la búsqueda. 5. Observar el mensaje.
Resultado esperado	Mensaje de búsqueda sin resultados ("There is no product that matches the search criteria.").
Resultado obtenido	"Your shopping cart is empty!" / "¡Tu carrito de la compra está vacío!"
Evidencia	evidencias/defectos/DR-001_TC-N04.png; tc-n04-falla-esperada.png
Estado / Issue	Abierto — GitHub Issue #3 (https://github.com/leotedd/Proyecto-QA-Tienda/issues/3)

El enunciado exige un mínimo de 7 defectos para el Grupo 10. A la fecha de este informe solo DR-001 está documentado con reporte, evidencia e Issue. El equipo observó además un candidato a defecto (las páginas de cuenta creada y cierre de sesión muestran el encabezado "Account" en lugar del título de la página), que no ha sido formalizado como reporte ni como Issue y, por tanto, no se contabiliza. Los defectos restantes quedan pendientes.

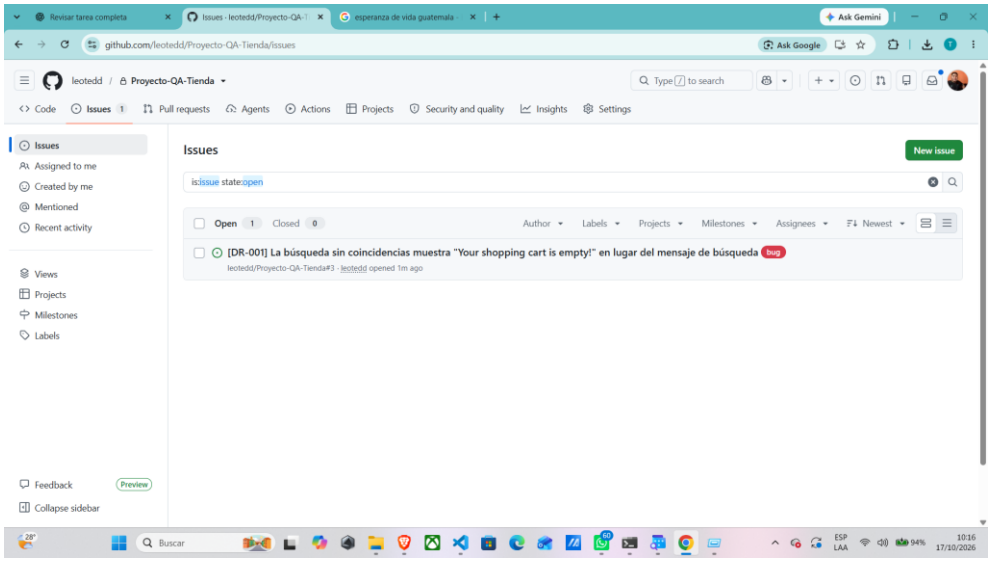


Figura 2. GitHub Issue #3 correspondiente al defecto DR-001.

13. Matriz de trazabilidad

Func.	Casos manuales	Pruebas automatizadas	Evidencia	Defecto
F01	TC-R01–R04 (pendientes)	regression-auth: 5 pruebas de registro	Capturas de suite	—
F02	TC-L01, L02, L04 (pendientes)	smoke: login; regression-auth: login válido y 3 rechazados	Capturas de suite	—
F03	TC-L03 (pendiente)	regression-auth: cierre de sesión	Capturas de suite	—
F04	TC-P01, P02 (pendientes)	regression-reviews: editar perfil	Capturas de suite	—
F05	TC-P03 (pendiente)	regression-reviews: cambiar contraseña	Capturas de suite	—
F06	TC-V01–V03 (Pass)	regression-reviews: TC-V01, V02, V03	TC-V01–V03.png	—
F07	TC-C01–C03 (Pass)	regression-reviews: TC-C01, C02, C03	TC-C01–C03.png	—
F08	TC-N01, N02 (Pass); N05, N06 (pend.)	smoke: inicio, TC-N01, TC-N02, detalle	TC-N01, N02.png	—
F09	TC-N03 (Pass), TC-N04 (Fail)	regression-reviews: TC-N03, TC-N04 (test.fail)	TC-N03, N04.png; DR-001	DR-001 / #3

Las nueve funcionalidades del alcance cuentan con cobertura automatizada; F06 a F09 cuentan además con cobertura manual ejecutada.

14. Evidencias de ejecución

Tipo	Ubicación
Capturas de casos manuales F06–F09	evidencias/casos-manuales/martinez/ (10 capturas)
Capturas de la suite (terminal y reporte HTML)	evidencias/automatizadas/teddy/ (7 capturas y README)
Defecto DR-001	evidencias/defectos/DR-001_TC-N04.png; reportes/defectos/Reporte_Defecto_DR-001_TC-N04.docx
Exploratorias	evidencias/exploratorias/martinez/EXP-05_categoria-software-sin-productos.png
GitHub Issues	evidencias/github-issues/issue-3-DR-001.png
Video de Milton Adrián Martínez Ávila	evidencias/videos/martinez/Ejecucion_Manual_Milton_F06-F09.mp4
Video de Teddy Leonardo Hernández Pérez	https://drive.google.com/file/d/12CX_3WNO-PgAgyHiW9NWq8i5yC-7peCI/view
Video de Gelen Dayanna López Morales	No se encontró registro en el repositorio (pendiente)

15. Análisis de resultados y estabilidad

Los resultados muestran que las funcionalidades en alcance se comportan según lo esperado en los tres motores de navegador, con una excepción consistente: el manejo del caso sin resultados en búsqueda y en categorías vacías (DR-001). La coincidencia entre la ejecución

manual, la exploratoria y la automatizada en tres navegadores refuerza que se trata de un defecto de la aplicación y no de un problema de la prueba.

En cuanto a estabilidad, la documentación del equipo identifica riesgos potenciales por inspección de código: un localizador posicional del mensaje de resultados de búsqueda, la dependencia del primer encabezado del contenido, el contador variable de la pestaña de reseñas, la dependencia de datos del catálogo compartido y las diferencias entre motores. Como factores a favor, la suite no usa esperas fijas, se apoya en el auto-waiting y en aserciones web, y aísla el estado mediante fixtures.

Durante el desarrollo se presentó una falla determinista el 10/10/2026: las pruebas de registro satisfactorio y de cierre de sesión esperaban un encabezado h1 que la aplicación mostraba como "Account". Se diagnosticó que la aplicación no muestra el título esperado en ese encabezado y se corrigió la verificación para usar el título de la página (toHaveTitle). Por su naturaleza consistente, esta falla no se clasifica como prueba inestable.

Adicionalmente, el 23/10/2026 se repitieron cinco veces, sin reintentos, ocho pruebas que no crean datos en Chromium: 35 de 35 ejecuciones aprobadas y 5 de 5 fallas esperadas de TC-N04, sin intermitencias. Se observó que el tiempo de respuesta del sitio varía hasta 2.6 veces entre corridas (por ejemplo, de 5.0 a 13.2 segundos), principal factor de riesgo de inestabilidad. Con los datos disponibles no se ha demostrado una prueba inestable real; por honestidad, el informe no declara haber corregido una, y este requisito del enunciado queda pendiente de evidencia.

16. Dificultades y soluciones

Dificultad	Solución aplicada
Entorno compartido con datos cambiantes	Usuarios únicos por ejecución con correos example.com y fixtures que crean su propio estado.
Encabezados de la aplicación que no coinciden con el título de la página	Verificación mediante el título de la página (toHaveTitle) en lugar del encabezado h1.
Defecto real que haría fallar la suite	Marcar TC-N04 con test.fail() y anotación del defecto, sin cambiar la expectativa.
Los reportes HTML se sobrescriben en cada corrida	Conservar capturas de terminal y del reporte HTML como evidencia versionada.
Seguridad de dependencias	.npmrc con ignore-scripts=true e instalación explícita de navegadores.
Numeración de casos diferente entre documentos	Realineación a la numeración oficial F01–F09 en la matriz de trazabilidad.

17. Conclusiones individuales

Las siguientes conclusiones son borradores elaborados a partir de las contribuciones documentadas de cada integrante y quedan pendientes de validación y reescritura personal por cada uno antes de la entrega definitiva.

17.1 Teddy Leonardo Hernández Pérez

[Borrador pendiente de validación personal] Mi participación se centró en la coordinación del repositorio y en la construcción de la suite automatizada. Implementar el patrón Page Object Model me permitió comprobar que separar los localizadores de la lógica de las pruebas no es solo una cuestión de orden: cuando la aplicación mostró un encabezado distinto al esperado en las páginas de cuenta, la corrección se resolvió en un punto concreto sin reescribir las pruebas. También aprendí a diferenciar con rigor una prueba inestable de un defecto real; TC-N04 falla siempre en la misma aserción y en los tres navegadores, por lo que lo correcto fue documentarlo como DR-001 y marcarlo como falla esperada, no modificar la expectativa para que pase. Como reflexión personal, reconozco que la ejecución multinavegador y el análisis de estabilidad requieren datos repetidos y verificables, y que la calidad de un proyecto depende tanto de la honestidad de sus reportes como de la cantidad de pruebas que contiene.

17.2 Gelen Dayanna López Morales

[Borrador pendiente de validación personal] Mi aporte consistió en elaborar el Plan de Aseguramiento de la Calidad, diseñar los casos funcionales de registro, sesión, perfil y contraseña, y construir la matriz de trazabilidad. Este trabajo me mostró que la planificación es la base que da sentido a las pruebas: definir alcance, criterios de entrada y salida, riesgos y plantillas permitió que el equipo hablara un mismo lenguaje. La matriz de trazabilidad evidenció qué funcionalidades tenían cobertura manual, automatizada y evidencias, y también qué faltaba, lo que considero su mayor valor. Como reflexión, reconozco que un caso diseñado no equivale a un caso ejecutado, y que el registro de resultados con evidencia es lo que convierte un plan en aseguramiento real de la calidad.

17.3 Milton Adrián Martínez Ávila

[Borrador pendiente de validación personal] Mi responsabilidad fue diseñar y ejecutar los casos manuales de calificación, reseña, navegación y manejo de errores, realizar las pruebas

exploratorias y reportar los defectos. Ejecutar los diez casos con evidencia me permitió observar el comportamiento real de la aplicación más allá de lo que describen los requisitos. La exploración libre fue especialmente valiosa: al revisar una categoría sin productos confirmé que el mensaje de carrito vacío aparecía también fuera de la búsqueda, lo que fortaleció el reporte de DR-001. Como reflexión, aprendí que un buen reporte de defecto debe ser reproducible, con pasos claros, evidencia y clasificación de severidad y prioridad, para que el equipo de desarrollo pueda actuar sobre él.

18. Recomendaciones

- Ejecutar y registrar con evidencia los 13 casos manuales de F01–F05 y F08 adicionales para cumplir el criterio de salida del 100 % de casos ejecutados.
- Completar la documentación de al menos seis defectos adicionales reales, cada uno con su GitHub Issue, para alcanzar el mínimo de siete del Grupo 10.
- Alinear playwright.config.ts con el ejemplo del enunciado: timeout de 30 segundos, un reintento, video y traza retenidos en fallos y reporter de lista y HTML sin apertura automática.
- Sustituir el localizador posicional del mensaje de resultados de búsqueda por uno anclado al encabezado de la búsqueda.
- Ampliar las pruebas smoke con el formulario de inicio de sesión visible y el cierre de sesión con redirección, como sugiere el enunciado.
- Ejecutar corridas repetidas en Firefox y WebKit para completar el análisis de estabilidad con datos reales.
- Conservar el último reporte HTML en la carpeta reportes/ y otorgar acceso al catedrático al repositorio privado.
- Como mejoras opcionales, incorporar pruebas de accesibilidad con axe-core y ejecución en GitHub Actions.

19. Referencias

Cantú, J. (2011). Desarrollo de una cultura de calidad (4.ª ed.). McGraw-Hill.

Galin, D. (2004). Software quality assurance: From theory to implementation. Pearson Education.

GitHub. (2026). Acerca de las propuestas (issues). GitHub Docs. https://docs.github.com/es/issues

International Software Testing Qualifications Board. (2026). Glosario ISTQB. https://www.istqb.org/

Kaner, C., Falk, J., y Nguyen, H. Q. (1999). Testing computer software (2nd ed.). Wiley.

Microsoft. (2026). Playwright: Getting started. https://playwright.dev/docs/intro

Microsoft. (2026). Playwright: Test projects. https://playwright.dev/docs/test-projects

Naik, K., y Tripathy, P. (2008). Software testing and quality assurance: Theory and practice. Wiley.

Toledo Rodríguez, F., et al. (2014). Introducción a las pruebas de sistemas de información. Abstracta.

Anexos

Anexo A. Enlaces del proyecto

Recurso	Enlace o ruta
Repositorio	https://github.com/leotedd/Proyecto-QA-Tienda
Issue DR-001	https://github.com/leotedd/Proyecto-QA-Tienda/issues/3
Video de Teddy Leonardo Hernández Pérez	https://drive.google.com/file/d/12CX_3WNO-PgAgyHiW9NWq8i5yC-7peCI/view
Video de Milton Adrián Martínez Ávila	evidencias/videos/martinez/Ejecucion_Manual_Milton_F06-F09.mp4
Casos manuales F06–F09	documentacion/casos-prueba/Casos_Prueba_Martinez_F06-F09.xlsx
Casos F01–F05	documentacion/casos-prueba/casos-funcionales-F01-F05.md
Casos automatizados	documentacion/casos-prueba/Casos_Prueba_Hernandez_Automatizacion.xlsx
Plan QA	documentacion/plan-qa/plan-aseguramiento-calidad.md
Matriz de trazabilidad	documentacion/trazabilidad/matriz-trazabilidad.md
Reporte de ejecución	reportes/ejecucion-automatizada.md

Anexo B. Capturas de ejecución

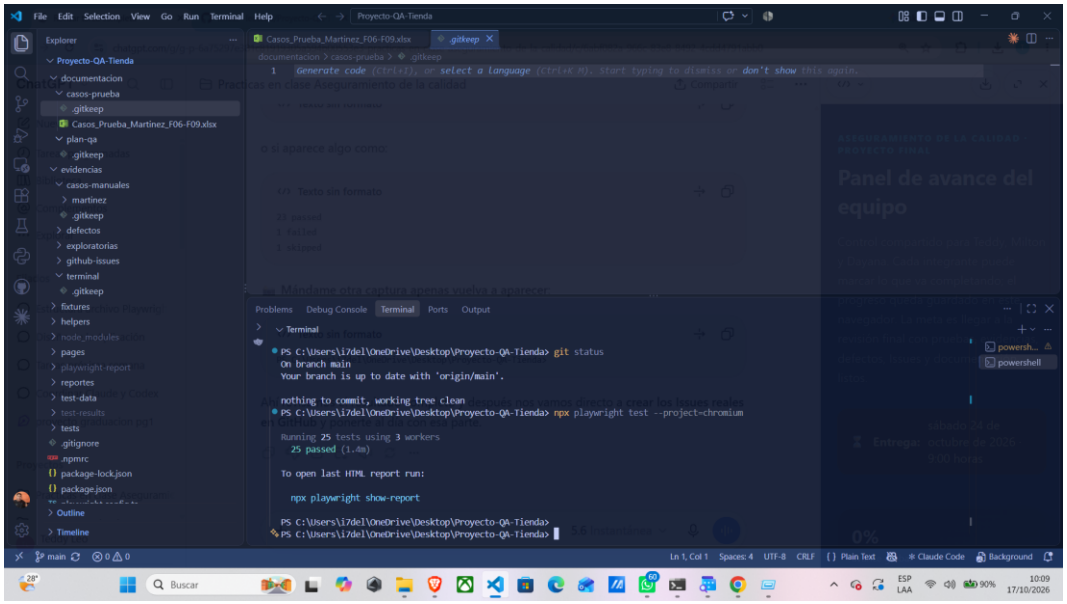


Figura B1. Ejecución en Chromium: 25 passed.

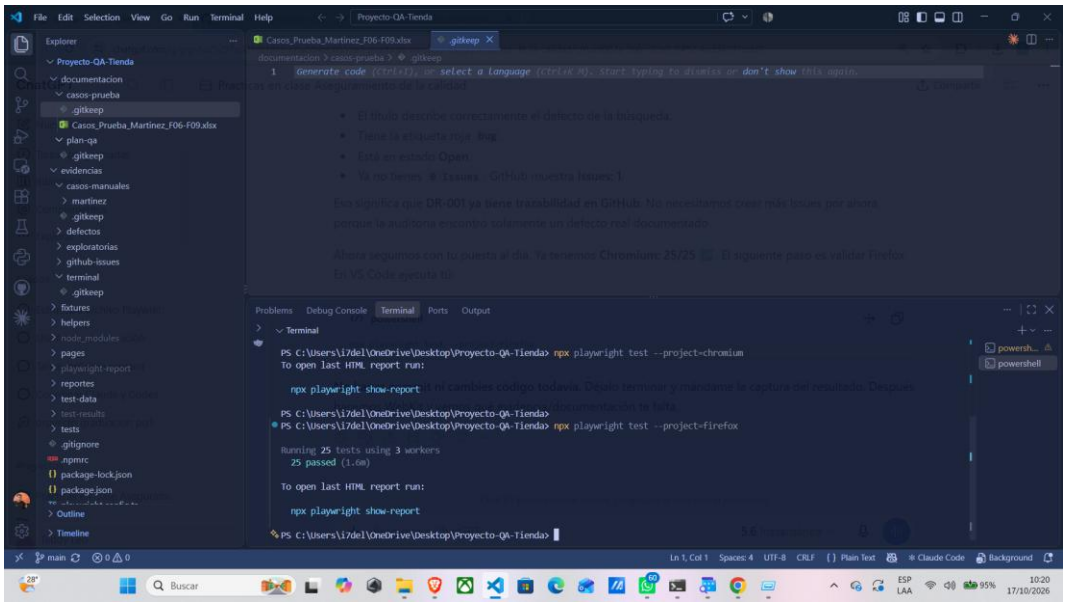


Figura B2. Ejecución en Firefox: 25 passed.

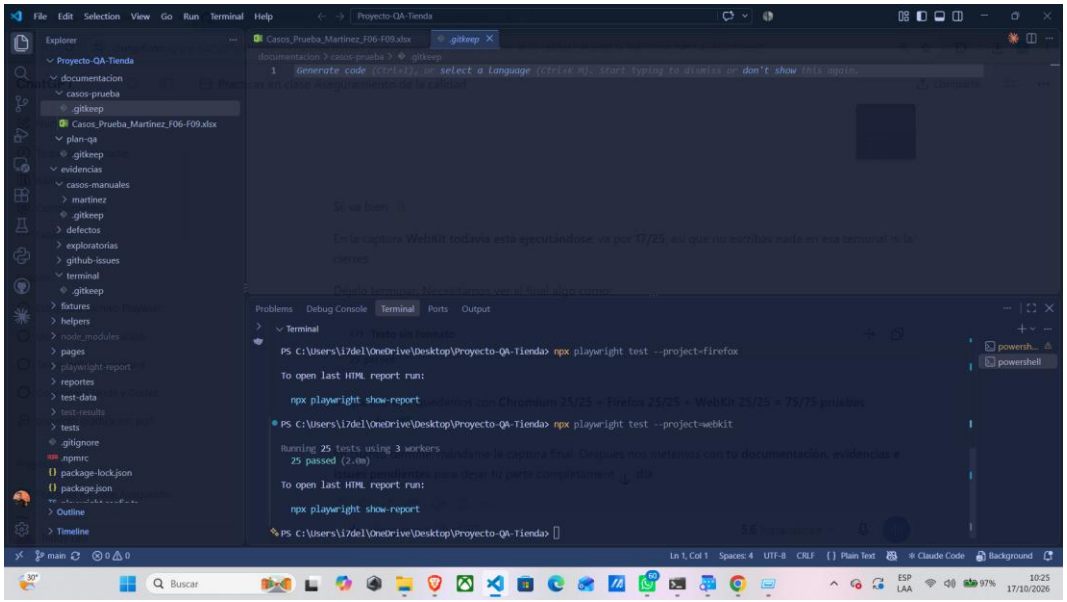


Figura B3. Ejecución en WebKit: 25 passed.

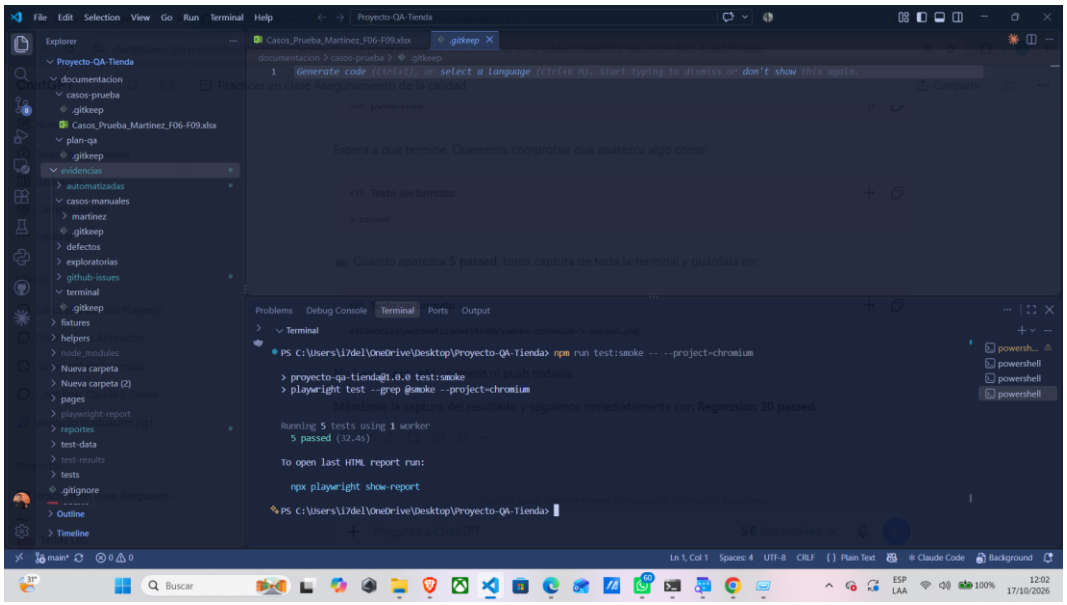


Figura B4. Ejecución selectiva @smoke: 5 passed.

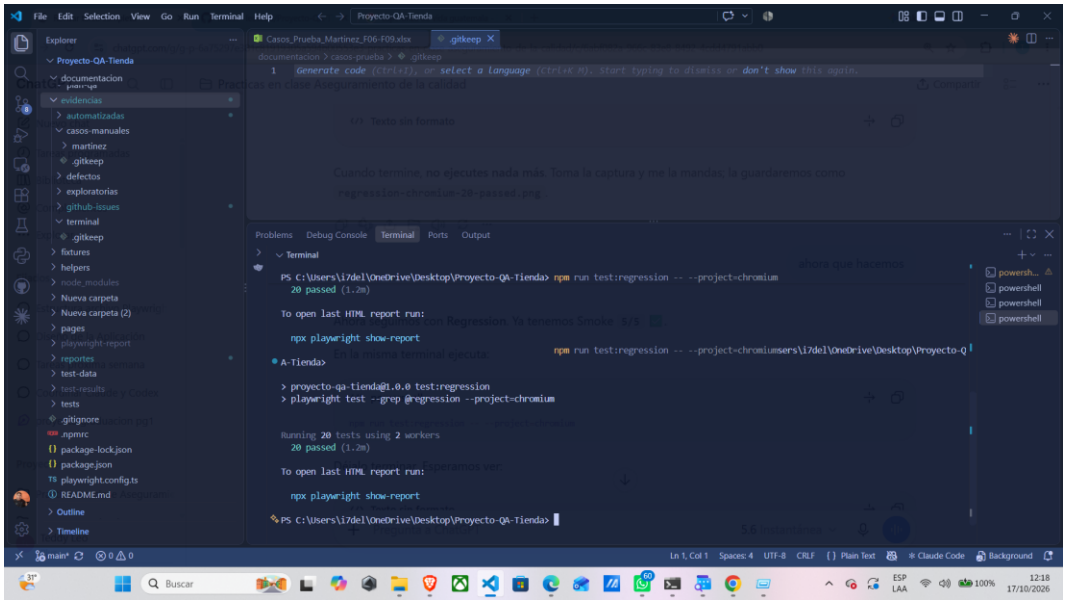


Figura B5. Ejecución selectiva @regression: 20 passed.

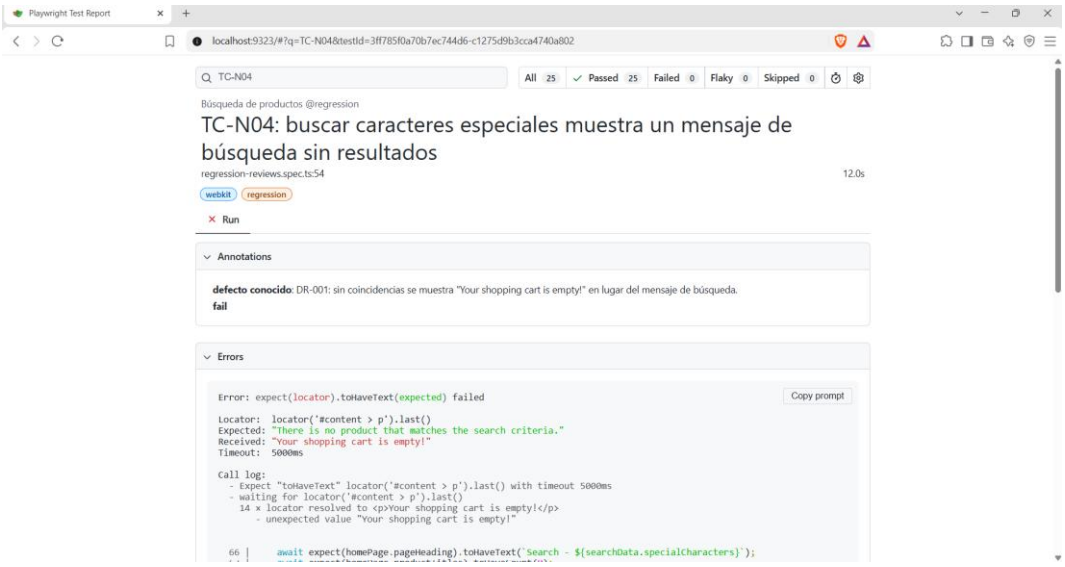


Figura B6. TC-N04 como falla esperada por DR-001.

Anexo C. Evidencias manuales y de defectos

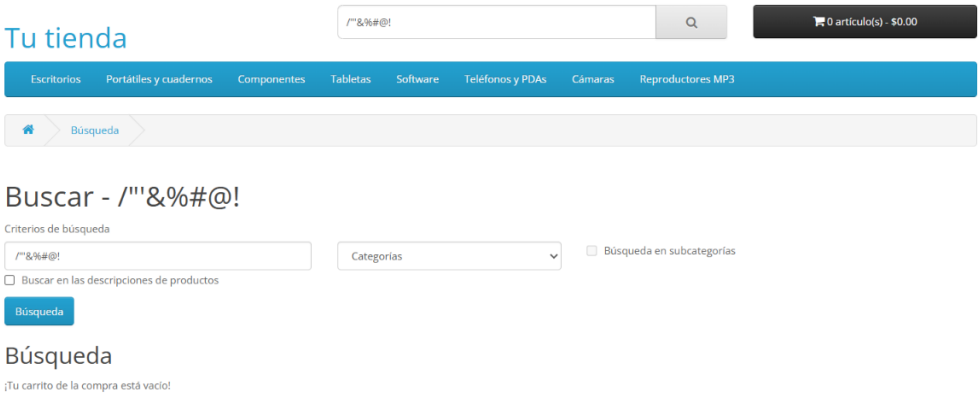


Figura C1. Evidencia manual de DR-001 (TC-N04).

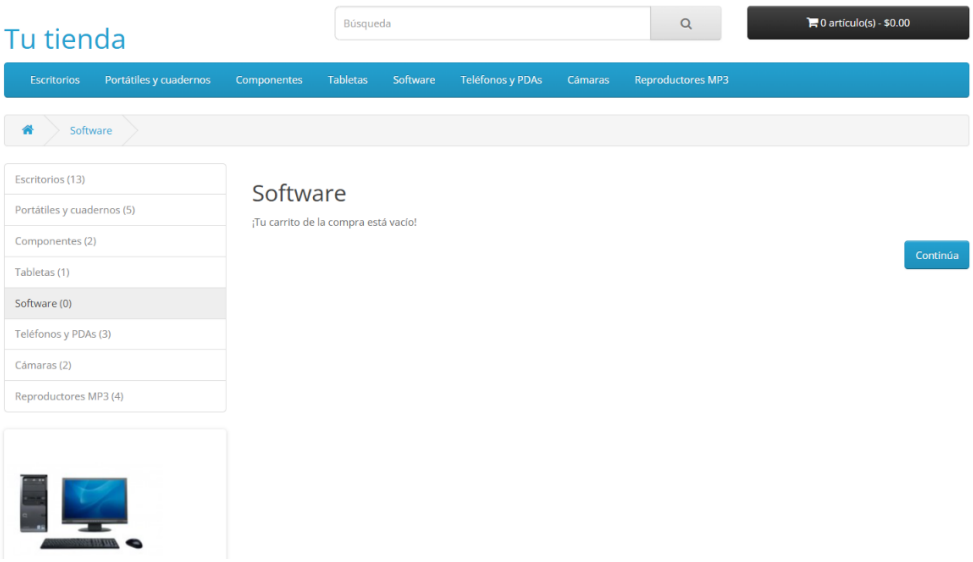


Figura C2. Exploratoria EXP-05: categoría Software sin productos.